

AI / ML Roadmap

First Year — Complete Study Plan

This roadmap is designed for first-year students who are starting from zero. It is divided into two semesters, each six months long, covering everything from basic Python programming and mathematics through to deep learning and model deployment. Following this plan consistently will give you a solid, practical foundation in Artificial Intelligence and Machine Learning by the end of the year.

Semester	Duration	Focus
Semester 1	Months 1–6	Python, Mathematics, Data Tools, ML Fundamentals
Semester 2	Months 7–12	Advanced ML, Deep Learning, NLP, Deployment, Capstone

What You Will Achieve by the End of Year 1

By following this roadmap, you will build a strong foundation that prepares you for internships, research roles, and advanced studies in AI/ML. Here is a summary of the key outcomes:

Hands-on Python Skills

You will be proficient in Python and its core data science libraries — NumPy, Pandas, Matplotlib, Scikit-learn, and PyTorch. You will be able to write clean, well-structured code and work comfortably inside Jupyter Notebooks.

Mathematical Intuition

You will understand the mathematics that drives ML algorithms — linear algebra for understanding model weights and transformations, calculus for gradient-based optimization, and probability/statistics for understanding model uncertainty and data distributions.

ML Algorithm Literacy

You will know how and when to apply a wide range of algorithms — from linear regression and decision trees to random forests, gradient boosting, and basic neural networks.

Deep Learning Introduction

You will understand how neural networks work, be able to build and train a simple network using PyTorch, and have practical experience with CNNs for image data and transformers for NLP.

Project Portfolio

You will have at least 3 complete ML projects on GitHub, including a capstone end-to-end project with proper documentation, demonstrating your skills to future employers or universities.

Deployment Basics

You will know how to package and serve a trained ML model as a REST API using Flask or FastAPI, which is a critical skill for taking ML from notebooks to real applications.

Semester 1 — Foundations & Core Programming

Months 1 through 6 | 6 modules | Focus: Build the bedrock skills before touching ML algorithms

Semester 1 is about building a strong foundation. Many students rush into neural networks without understanding Python or the underlying math, and this leads to confusion later. Take your time with each module below. The goal is not to rush — it is to understand deeply.

Programming · Months 1–2

Python for AI/ML

Python is the universal language of AI and ML. You need to be comfortable with it before anything else. Focus on writing clean code, not just getting things to run.

- Variables, data types, loops, and conditionals
 - Functions, scope, and recursion basics
 - Lists, tuples, dictionaries, and set comprehensions
 - Object-Oriented Programming — classes, objects, inheritance
 - File I/O, exception handling, and context managers
 - Virtual environments, pip package management, and Jupyter Notebooks
-

Mathematics · Months 1–3 (parallel)

Math Essentials for ML

You do not need a maths degree, but you do need a working understanding of two areas: linear algebra and probability. These form the language of every ML algorithm.

- Linear algebra — vectors, matrices, and how they represent data
 - Matrix multiplication, dot products, and transpose operations
 - Gradient descent — the optimisation method behind nearly all of ML
 - Probability — conditional probability, distributions (Normal, Bernoulli)
 - Statistics — mean, variance, standard deviation, and Bayes' theorem
-

Data Tools · Month 2–3

NumPy & Pandas

NumPy and Pandas are the two most important libraries for working with data in Python. NumPy handles numerical computation; Pandas handles tabular data.

- NumPy arrays — creation, indexing, slicing, and broadcasting
 - Vectorised operations (much faster than Python loops)
 - Pandas Series and DataFrames — the core data structures
 - Loading data from CSV, Excel, and JSON files
 - Cleaning data — handling missing values, duplicates, and type errors
 - Filtering, groupby, aggregations, merging, and reshaping datasets
-

Matplotlib & Seaborn

Data visualisation is an essential part of the ML workflow. Before modelling, you must explore your data visually to understand its structure and distributions.

- Line plots, bar charts, scatter plots, and pie charts
 - Histograms and box plots for understanding distributions
 - Heatmaps for visualising correlations between features
 - Pair plots for spotting relationships between multiple variables
 - Exploratory Data Analysis (EDA) — a structured approach to understanding datasets
 - Customising plots — labels, titles, legends, colour palettes, and figure sizes
-

ML Fundamentals

Before writing a single line of Scikit-learn code, you must understand the core concepts behind machine learning. This module is all theory — but it is crucial.

- Supervised learning (labelled data) vs unsupervised learning (no labels)
 - Regression tasks (predicting a number) vs classification tasks (predicting a category)
 - Training set, validation set, and test set — why you need all three
 - Overfitting (model too complex) and underfitting (model too simple)
 - The bias-variance tradeoff — the central tension in all of ML
 - Loss functions — Mean Squared Error for regression, Cross-Entropy for classification
 - Gradient descent and how optimizers update model parameters
-

Scikit-learn & Classic Algorithms

Now you put theory into practice. Scikit-learn is the standard Python library for classical machine learning, with a consistent and easy-to-use API.

- Linear regression — predicting continuous values with a straight line
 - Logistic regression — binary classification using a sigmoid function
 - Decision trees — rule-based models that split data on feature thresholds
 - K-Nearest Neighbours (KNN) — classifying by similarity to neighbours
 - Building model pipelines — combining preprocessing and model into one object
 - Hyperparameter tuning with GridSearchCV and RandomizedSearchCV
 - Evaluation metrics — accuracy, precision, recall, F1-score, ROC-AUC
 - K-fold cross-validation — a robust way to estimate model performance
-

Semester 1 Goals

By the end of Month 6, you should be able to demonstrate the following:

✓ **Write clean Python scripts and Jupyter notebooks independently**

You should be able to set up a project, create a virtual environment, and write structured, readable Python code without relying on tutorials.

✓ **Solve basic linear algebra problems by hand and with NumPy**

Understand matrix multiplication, dot products, and vector norms both conceptually and in code.

✓ **Load, clean, and visualise a real dataset**

Work through a real dataset (e.g. Titanic or Iris) — handle missing values, encode categoricals, and produce meaningful EDA plots.

✓ **Train and evaluate at least two ML models**

Train a logistic regression and a decision tree; compare their performance using precision, recall, and F1-score.

✓ **Understand and implement k-fold cross-validation**

Know why a single train/test split can be misleading and how cross-validation gives a more reliable performance estimate.

✓ **Complete a beginner Kaggle competition end-to-end**

Submit a scored entry on a beginner competition such as Titanic Survival Prediction — this proves you can handle a real ML pipeline.

Semester 2 — Applied ML & Deep Learning

Months 7 through 12 | 6 modules | Focus: Go deeper, build real projects, and learn modern tools

Semester 2 assumes you are comfortable with Python, understand basic ML concepts, and have worked through at least one end-to-end dataset. Now you will go deeper — into ensemble methods, neural networks, computer vision, NLP, and real-world deployment. By the end, you will have a portfolio that demonstrates genuine, employable skills.

ML (Advanced) · Month 7

Advanced Algorithms & Ensemble Methods

Classical ML still powers a huge portion of production ML systems. Ensemble methods like random forests and gradient boosting consistently outperform simpler models on structured/tabular data. Learn these well before jumping to deep learning.

- Ensemble learning — why combining weak learners produces stronger models
- Bagging and random forests — decorrelated trees through feature subsampling
- Gradient boosting — building trees sequentially to correct prior errors
- XGBoost and LightGBM — the industry-standard gradient boosting libraries
- Support Vector Machines (SVMs) — maximum-margin classifiers
- Kernel methods — using the kernel trick to handle non-linear data
- Unsupervised learning — K-Means and DBSCAN clustering algorithms
- Principal Component Analysis (PCA) — reducing dimensionality while preserving variance

Deep Learning · Months 8–9

Neural Networks from Scratch

Deep learning is the backbone of modern AI. Start by understanding the fundamentals — how neurons work, how networks learn, and how to implement this in PyTorch. Avoid the black-box mindset: understand what is happening inside.

- The perceptron — the building block of all neural networks
 - Activation functions — ReLU, sigmoid, tanh, and when to use each
 - Fully connected (dense) networks — layers, weights, biases, and forward pass
 - Backpropagation — computing gradients through the chain rule
 - Loss functions for deep learning — cross-entropy, MSE, and custom losses
 - Optimizers — SGD, Adam, RMSProp and their hyperparameters
 - PyTorch basics — tensors, autograd, nn.Module, and DataLoader
 - Training loops — batching, epochs, shuffling, and monitoring loss curves
 - Regularisation — dropout, weight decay, and batch normalisation
-

CNNs & Image Data

Convolutional Neural Networks (CNNs) are the standard architecture for any task involving images. Understanding CNNs will also help you understand the intuition behind attention and transformers later.

- Convolution operation — filters, feature maps, and stride
 - Pooling layers — max pooling and average pooling for spatial downsampling
 - Classic CNN architectures — LeNet, AlexNet, and modern ResNet
 - Transfer learning — using pretrained models (ResNet, EfficientNet) on new tasks
 - Fine-tuning vs feature extraction — when to use each approach
 - Image preprocessing — normalisation, resizing, and channel ordering
 - Data augmentation — flipping, cropping, rotation, and colour jitter
 - Practical project — image classifier on MNIST or CIFAR-10
-

Introduction to Natural Language Processing

NLP is one of the most active areas of AI research. Start with the fundamentals — text preprocessing and classical NLP — before moving to modern transformers.

- Text preprocessing — tokenisation, lowercasing, stopword removal, and stemming
 - Bag-of-Words (BoW) model — representing text as word frequency vectors
 - TF-IDF — weighting terms by their importance across a corpus
 - Word embeddings — Word2Vec and GloVe for dense vector representations
 - Text classification project — sentiment analysis or spam detection
 - Introduction to the Transformer architecture and self-attention
 - Using HuggingFace Transformers — loading and fine-tuning a pretrained BERT model
-

ML Workflow, Tooling & Deployment

Building good models is only half the job. You also need to track experiments, version your code, and serve your models. These skills are what separate academic ML from production-ready ML.

- Git and GitHub — version control for ML code and notebooks
 - Experiment tracking with MLflow — logging parameters, metrics, and artifacts
 - Model serialisation — saving and loading models with pickle and joblib
 - Reproducibility — setting random seeds, pinning dependency versions
 - Model evaluation and selection — comparing experiments systematically
 - Building a REST API with Flask — wrapping your model in a web endpoint
 - FastAPI as a modern alternative — async endpoints and automatic docs
 - Docker basics — containerising your ML application (introductory awareness)
-

End-to-End ML Project

The capstone project is the most important deliverable of the year. It must be a complete, real ML project that you designed and executed independently. This is what you will show to employers, universities, and collaborators.

- Problem definition — choosing a meaningful problem with real-world relevance
 - Data collection and storage — scraping, APIs, or curated open datasets
 - Full exploratory data analysis (EDA) with visualisations
 - Feature engineering — creating new features from raw data
 - Model selection — training and comparing at least 3 different models
 - Hyperparameter tuning and final model evaluation on a held-out test set
 - Model deployment — serving predictions via a simple REST API
 - Write-up and documentation — a clear README and a short project report
 - GitHub portfolio — pushing clean, well-commented code and a notebook walkthrough
-

Semester 2 Goals

By the end of Month 12, you should be able to demonstrate the following:

✓ **Implement and compare ensemble models**

Train a Random Forest and XGBoost on the same dataset; tune hyperparameters and explain the difference in performance.

✓ **Build and train a neural network from scratch in PyTorch**

Write a training loop without relying on high-level wrappers — forward pass, loss computation, backprop, and optimizer step.

✓ **Complete an image classification project using CNNs**

Achieve over 90% accuracy on MNIST or over 75% on CIFAR-10; use data augmentation and transfer learning.

✓ **Build a text classifier using TF-IDF or Word2Vec**

Train a sentiment or topic classifier on a real text dataset and evaluate it with classification report metrics.

✓ **Deploy a model as a REST API**

Serve your trained model via a Flask or FastAPI endpoint that accepts a JSON input and returns a prediction.

✓ **Publish a complete end-to-end ML project on GitHub**

Your repository should include clean code, a Jupyter notebook walkthrough, a requirements.txt, and a clear README.

Final Goals — End of Year 1

The table below summarises what you should have accomplished, know, and be able to demonstrate at the end of the full 12-month programme.

Area	Expected Outcome
Portfolio	3 or more ML projects on GitHub, including one full end-to-end capstone project with a write-up and a deployed API endpoint.
Algorithms	Comfortable implementing linear models, decision trees, ensemble methods (Random Forest, XGBoost), and a basic fully connected neural network from scratch.
Python Proficiency	Proficient with NumPy, Pandas, Matplotlib, Scikit-learn, and PyTorch. Able to write clean, modular Python code with proper project structure.
Mathematical Foundation	Understand gradients, matrix operations, and probability distributions well enough to read and follow most introductory ML papers.
Kaggle Experience	At least one ranked submission on a Kaggle competition (e.g. Titanic, House Prices). Familiar with competition workflow and leaderboard evaluation.
Model Deployment	Able to take a trained Scikit-learn or PyTorch model, serialise it, wrap it in a REST API using Flask or FastAPI, and run it locally.
Deep Learning Basics	Understand how CNNs work and have applied transfer learning (ResNet or similar) to an image classification task with data augmentation.
NLP Fundamentals	Familiar with text preprocessing, TF-IDF, word embeddings, and the basics of the Transformer architecture; has used HuggingFace Transformers at least once.

Study Tips & How to Use This Roadmap

Consistency over intensity

Studying for 1 to 2 hours every day is far more effective than 10-hour sessions on weekends. Build a daily habit — even 45 minutes of focused practice compounds quickly over 12 months.

Code everything yourself

Reading or watching tutorials is not the same as learning. After every concept, close the tutorial and re-implement it from memory. Only hands-on practice builds real understanding.

Do not skip the mathematics

Many students skip the math and learn ML as a collection of API calls. This works until it doesn't — when debugging fails or when you need to understand why a model is not converging. Invest time in linear algebra and calculus in Semester 1.

Work with real data early

Kaggle, UCI ML Repository, and HuggingFace Datasets all have freely available real-world datasets. Using messy, real data instead of clean toy datasets prepares you for actual work.

Build in public

Push your code and notebooks to GitHub regularly. Write short blog posts or Notion pages explaining what you learned. Teaching others forces you to understand deeply.

Track your progress

Use a simple checklist or Notion board to mark topics as 'learning', 'understood', or 'practiced'. Revisit topics that are marked 'understood' at least once every two weeks.

Remember: the goal of Year 1 is not to know everything — it is to build a solid, practical foundation and develop the habit of learning continuously. Every expert was once a beginner. Follow the plan, stay consistent, and the results will follow.